

猫集闲置商品交易平台

软件功能说明书

目录

- 项目设计思路.....1
 - 一、 项目题目.....1
 - 二、设计的主要技术指标.....1
 - 三、系统主要功能模块.....1
 - 四、创新点.....1
- 1.系统简介.....3
- 2.需求分析.....4
 - 2.1 可行性分析.....4
 - 2.2 需求分析.....5
- 3 系统设计.....14
 - 3.1 架构设计.....14
 - 3.2 功能模块设计.....14
- 4 系统实现.....16
 - 4.1 开发环境.....16
 - 4.2 功能模块实现.....16

项目设计思路

一、项目题目

猫集闲置商品交易平台

二、设计的主要技术指标

1、建立运行环境：

①建站环境： Windows 10

②数据库平台： MySQL 8.0+

2、开发软件：

前端开发工具： trae

后端开发工具： IntelliJ IDEA、 trae

3、系统要求：

后端要求： JDK 17+、 Node.js 20.19+、 Maven 3.8+、 Redis 6.0+

浏览器： Chrome/Edge/Firefox

三、系统主要功能模块

1、用户模块

2、商品模块

3、订单模块

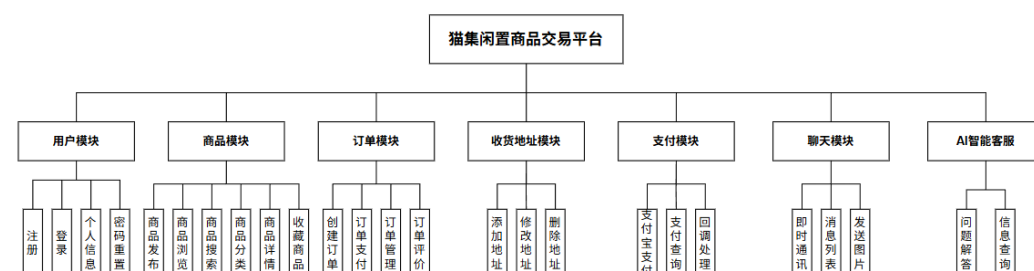
4、地址模块

5、支付模块

6、聊天模块

7、AI 客服模块

系统功能模块思维导图



四、创新点

1、AI 智能客服

平台使用 Spring AI 框架集成 DeepSeek 大语言模型，构建了名为"罐头"的 AI 智能客服系统。不同于简单的关键词匹配机器人，智能客服通过系统提示词（System Prompt）精准理解平台业务——能回答发布闲置、下单购买、订单查询、物流追踪、支付问题等场景化问题，并自动生成带直达链接的引导回复（如"点击这里发布闲置 → /personal-center/release-idle"）。前端对 AI 回复中的 URL 自动识别并渲染为可点击链接，内部链接采用 Vue Router 无刷新跳转，外部链接新窗口打开，兼顾安全性与用户体验。

2、物流轨迹地图可视化

对接快递 100 实时查询 API，获取快递物流轨迹后，不仅以时间线形式展示物流节点，更结合高德地图（AMap）进行可视化呈现。系统内置 30+ 城市经纬度坐标库，自动将物流节点的地理位置解析为地图标记，绘制出完整的"出发地→途经城市→目的地"运输路线，让用户直观了解包裹的实时位置和运输路径。

1.系统简介

猫集闲置商品交易平台是一个基于 **Spring Boot** 和 **Vue 3** 的电商平台，提供便捷的二手商品交易服务。系统采用前后端分离架构，前端使用 **Vue 3 + Vite + Element Plus** 构建响应式用户界面，后端采用 **Spring Boot 3 + MyBatis-Plus + Redis** 提供高性能 **RESTful API** 服务。

系统核心功能包括：用户注册登录、商品浏览与搜索、商品发布与管理、在线下单、支付宝支付集成、即时通讯聊天、收货地址管理、订单管理等模块。创新性地集成了 **Spring AI + deepseek** 智能客服，并采用 **JWT** 进行安全认证，结合高德地图（**AMap**）进行可视化呈现。

该系统的意义在于：为闲置物品流通提供规范化平台，减少资源浪费，促进资源利用效率提升；通过便捷的在线交易流程，提升生活便利性；集成即时通讯功能，方便买卖双方实时沟通；采用现代化技术栈，具有良好的可扩展性和维护性，为二手交易应用提供了可参考的实践案例。

2.需求分析

2.1 可行性分析

2.1.1 经济可行性分析

在开发猫集闲置商品交易平台的过程中,选用了多种免费且开源的技术和工具,包括 trae、IntelliJ IDEA 等免费开发环境,Vue 3、Spring Boot、MySQL、Redis 等开源框架,以及支付宝沙箱、邮件服务、高德地图、快递 100 等免费第三方服务。部署方面可采用低成本云服务器,配合 NATAPP 免费版实现内网穿透,大幅降低初期投入。系统建成后,可提供便捷的二手交易服务,降低生活成本,提高闲置物品利用率,具有显著的社会效益和潜在经济价值。开发过程中主要成本为人力与时间成本,但无需支付软件授权费用,整体成本可控。

综上所述,猫集闲置商品交易平台在经济上是可行的。

2.1.2 技术可行性分析

猫集闲置商品交易平台采用前后端分离架构,前端基于 Vue 3 + Vite + Element Plus 构建,技术成熟且社区活跃,开发人员在学校学习过程中已掌握 Vue 框架、JavaScript、CSS 等前端技术,并具备实际项目开发经验。后端采用 Spring Boot 3 + MyBatis-Plus + Redis 技术栈,这是业界广泛使用的成熟方案,开发团队对 Java Web 开发、RESTful API 设计、数据库操作等技术有扎实基础。数据库选用 MySQL 8.0。系统集成 WebSocket 实现实时通讯、支付宝 SDK 实现支付功能、Spring AI + deepseek 实现智能客服,这些技术均有完善的官方文档和丰富的实践案例。开发过程中使用的 Maven 依赖管理、SpringDoc API 文档等工具,均为成熟稳定的开发辅助技术。

综上所述,猫集闲置商品交易平台在技术上是可行的。

2.1.3 操作可行性分析

猫集闲置商品交易平台用户界面采用 Element Plus 组件库构建,界面简洁直观,操作流程清晰。用户无需特殊技能,只需使用主流浏览器(Chrome、Edge、Firefox 等)访问网站即可完成商品浏览、搜索、下单、支付、聊天等操作。商品发布流程简单,支持图片上传和文本描述,符合用户日常使用习惯。即时通讯功能采用类似微信的聊天界面设计,用户上手零门槛。个人中心提供统一的交易管理入口,包括发布闲置、我的发布、购买记录、收货地址管理、账户设置等功能模块,操作逻辑清晰。系统对浏览器无特殊要求,用户具备基本的上网技能即可熟练使用。

综上所述，猫集闲置商品交易平台在操作上是可行的。

2.2 需求分析

2.2.1 关键技术

前端采用 Vue 3 框架、JavaScript 编程语言、Element Plus 组件库，后端运用了 Spring Boot 3 框架、Java 编程语言、MyBatis-Plus 数据库访问框架、Redis 缓存数据库、MySQL 8.0 关系数据库等技术搭建二手猫二手交易网站，开发工具使用了 IntelliJ IDEA、Trae、Navicat 软件。操作环境是 Windows 10 系统，运行此项目的浏览器是 Google Chrome。

1、Vue 3 框架

Vue 3 是用于构建用户界面的渐进式 JavaScript 框架，采用组件化开发模式，具有响应式数据绑定、虚拟 DOM、组合式 API 等特性。Vue 3 基于 Proxy 实现了更强大的响应式系统，相比 Vue 2 在性能和类型推断方面都有显著提升。它支持单文件组件（SFC）开发模式，将模板、脚本和样式封装在同一个 .vue 文件中，提高了代码的可维护性和复用性。

2、JavaScript 编程语言

JavaScript 是一种基于原型和动态类型的脚本语言，是 Web 开发的核心技术之一。它支持面向对象、命令式、函数式等多种编程范式，具有事件驱动、异步编程、闭包等特性。现代 JavaScript(ES6+)引入了箭头函数、类、模块、Promise、async/await 等语法，大大提升了开发效率和代码可读性。

3、Element Plus 组件库

Element Plus 是一套基于 Vue 3 的桌面端组件库，提供了丰富的 UI 组件，包括表单、表格、对话框、导航、消息提示等。它采用 TypeScript 编写，支持完整的类型推导，具有简洁优雅的设计风格、完善的文档和活跃的社区支持。Element Plus 帮助开发者快速构建美观、一致的用户界面。

4、Spring Boot 3 框架

Spring Boot 3 是基于 Spring Framework 6 的快速开发框架，采用约定优于配置的理念，简化了 Spring 应用的创建和部署。它支持自动配置、起步依赖、内嵌服务器等特性，可以快速创建独立运行的 Spring 应用程序。Spring Boot 3 原生支持 Java 17+，提供了更好的性能优化、GraalVM 原生镜像支持、Observability 可观测性等现代特性。

5、MyBatis-Plus 数据库访问框架

MyBatis-Plus 是在 MyBatis 基础上扩展的持久层框架，它增强了 MyBatis 的功能，提供了通用的 CRUD 操作、代码生成器、分页插件、乐观锁插件等实用功能。MyBatis-Plus 遵循 JPA 规范，支持 Lambda 表达式查询、条件构造器、

多租户数据隔离等高级特性，大大简化了数据库操作代码的编写。

6、MySQL 8.0 数据库

MySQL 8.0 是 Oracle 公司推出的开源关系型数据库管理系统，具有高性能、高可靠性、易用性等特点。它支持 ACID 事务、外键约束、存储过程、触发器、视图等功能，提供了 JSON 数据类型、窗口函数、公共表表达式（CTE）等现代数据库特性。MySQL 采用 SQL 作为查询语言，支持多种存储引擎，适用于各种规模的应用系统。

7、Redis 缓存数据库

Redis 是一个开源的内存数据结构存储系统，常用作数据库、缓存和消息中间件。它支持字符串、哈希、列表、集合、有序集合等多种数据结构，提供了发布订阅、事务、Lua 脚本、持久化等功能。Redis 具有极高的读写性能，常用于缓存热点数据、分布式锁、会话管理、计数器等场景，有效减轻数据库压力。

8、WebSocket 实时通信技术

WebSocket 是一种在单个 TCP 连接上进行全双工通信的协议，它使得服务器能够主动向客户端推送数据。相比传统的 HTTP 轮询方式，WebSocket 具有更低的延迟和更高的效率，特别适用于实时聊天、在线游戏、股票行情等需要实时数据更新的场景。HTML5 标准原生支持 WebSocket API，前端可以方便地建立和维护 WebSocket 连接。

9、Spring AI AI 工程应用框架

Spring AI 是一个用于 AI 工程的应用框架。其目标是将 Spring 生态系统设计原则应用于 AI 领域，如可移植性和模块化设计，并推广将 POJO 作为应用构建模块到人工智能领域的应用。

2.2.2 业务流程分析

1、商品浏览/搜索业务

用户进入系统首页后，可选择搜索商品或浏览商品列表。如果选择搜索，输入关键词后系统显示搜索结果；如果选择浏览，系统按分类展示商品列表。用户可查看商品详情，业务流程图如图 2.1 所示。

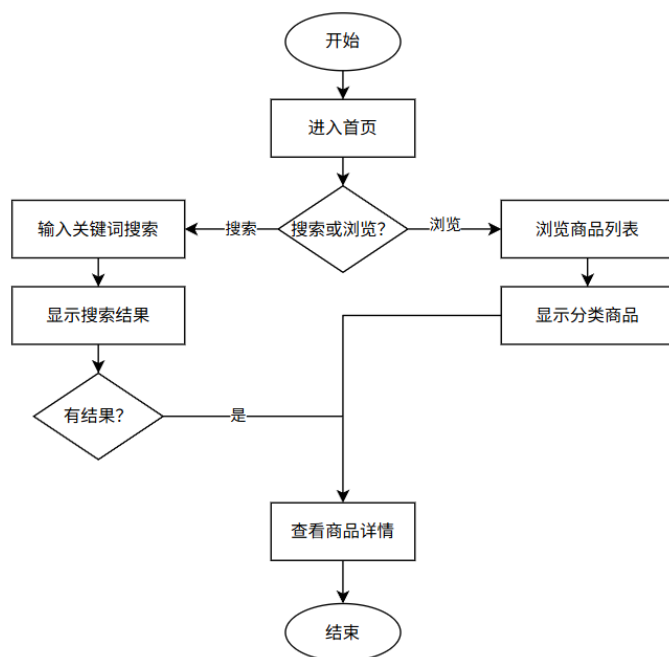


图 2.1 商品浏览/搜索业务流程

2、商品发布业务

用户进入发布闲置页面后，填写商品名称、价格、描述等信息，可选择上传图片文件，然后选择商品分类并提交发布。系统判断发布是否成功，成功后商品显示在"我的发布"列表中，业务流程图如图 2.2 所示。

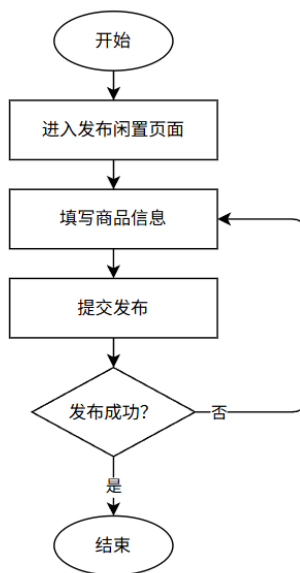


图 2.2 商品发布业务流程

3、订单支付业务

用户在商品详情页确认购买后，系统创建订单，用户选择收货地址并调用支付宝完成支付。系统判断支付是否成功，成功后更新订单状态并通知卖家已付款，业务流程图如图 2.3 所示。

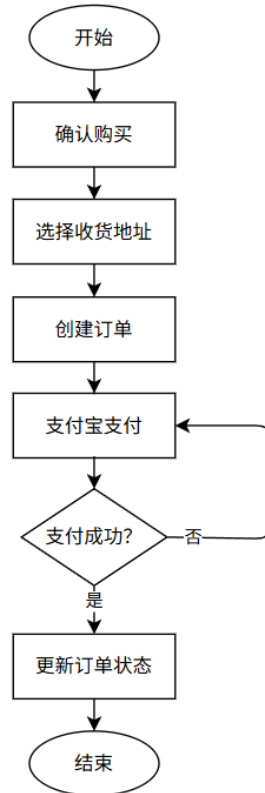


图 2.3 订单支付业务流程

4、即时通讯业务

用户在商品详情页点击"聊一聊"按钮，系统判断与该卖家的会话是否存在，不存在则创建新会话。进入聊天窗口后建立 WebSocket 连接，用户可发送和接收消息。当用户退出聊天时关闭 WebSocket 连接，业务流程图如图 2.5 所示。

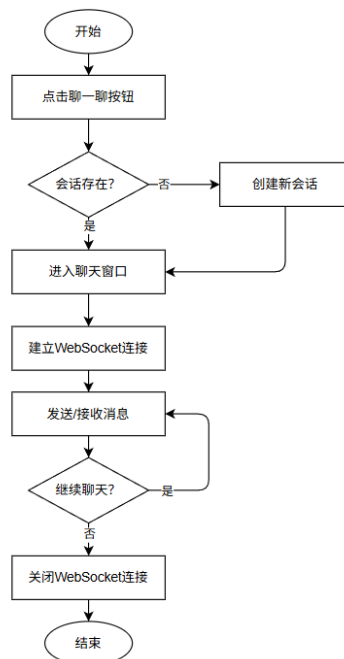


图 2.4 即时通讯业务流程

2.2.3 功能需求分析

本系统实现了用户模块、商品模块、订单模块、收货地址模块、支付模块、聊天模块、AI 智能客服模块等，具体的功能如下：

1、用户模块

用户可进行注册、登录、密码重置和个人信息修改等操作。用户可在个人中心修改昵称、头像等个人信息。用例图如图 2.5 所示。

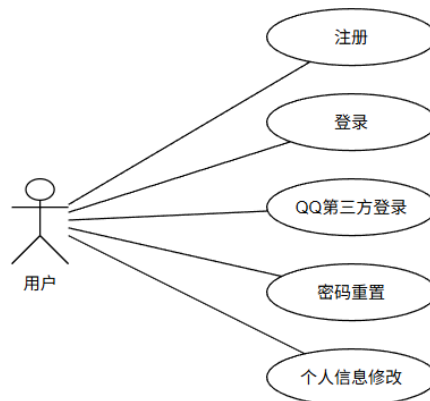


图 2.5 用户模块用例图

2、商品模块

用户可浏览首页商品列表、通过关键词搜索商品、按分类筛选商品、查看商品详情、发布闲置商品和收藏商品。商品发布支持上传图片、填写商品名称、价格、描述和选择分类。用例图如图 2.6 所示。

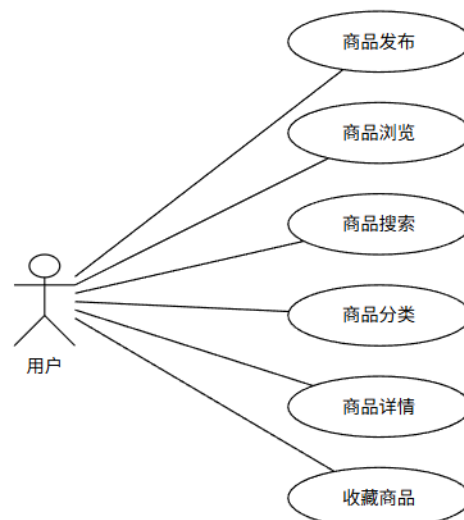


图 2.6 商品模块用例图

3、订单模块

用户可在商品详情页创建订单、选择收货地址后完成订单支付，在个人中心查看和管理自己的订单（包括购买记录和发布记录）。用例图如图 2.7 所示。

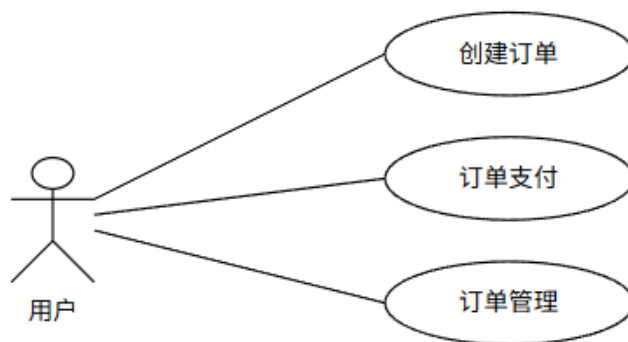


图 2.7 订单模块用例图

4、收货地址模块

用户可添加、修改和删除收货地址，在下单时选择已保存的收货地址。用例图如图 2.9 所示。

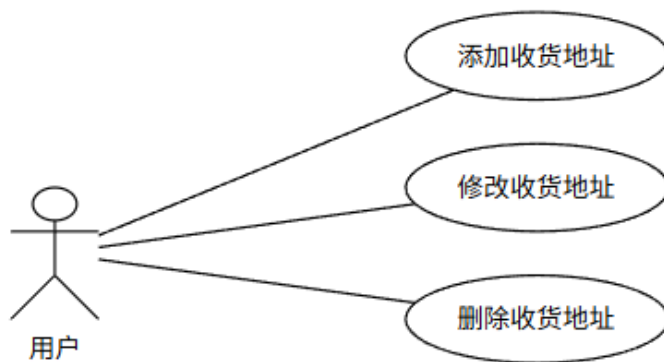


图 2.9 地址模块用例图

5、支付模块

系统集成支付宝支付功能，用户可通过支付宝完成订单支付，系统支持支付状态查询和异步回调处理，确保订单状态正确更新。用例图如图 2.10 所示。

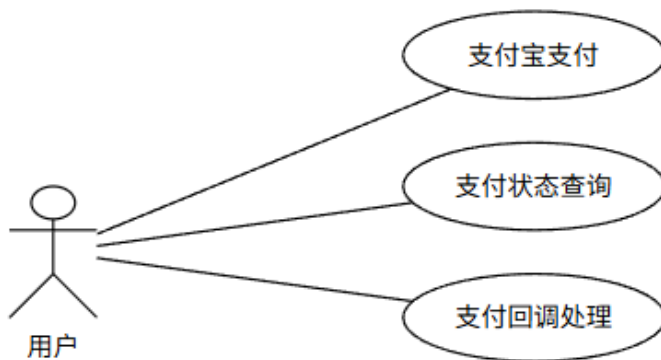


图 2.10 支付模块用例图

6、聊天模块

用户可在商品详情页点击"聊一聊"与卖家进行即时通讯，系统支持发送文本消息、表情和图片消息，消息通过 WebSocket 实时推送，并带有未读消息标记功能。用例图如图 2.8 所示。

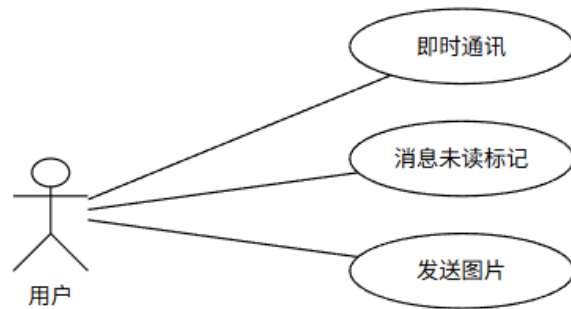


图 2.8 聊天模块用例图

7、AI 智能客服模块

系统集成支付宝支付功能，用户可通过支付宝完成订单支付，系统支持支付状态查询和异步回调处理，确保订单状态正确更新。用例图如图 2.10 所示。

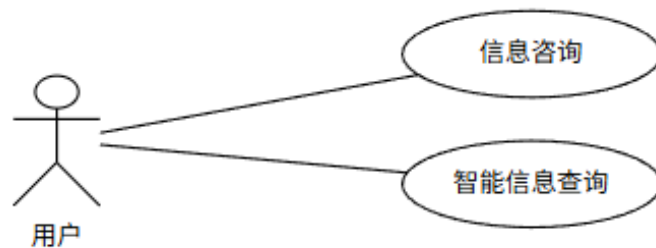


图 2.9 AI 智能客服模块用例图

2.2.4 非功能性需求分析

1、兼容性

系统前端基于 Vue.js 和 Element Plus 开发，支持主流浏览器（Chrome、Firefox、Edge、Safari 等）的正常使用。在不同操作系统（Windows、macOS、Linux）下，系统的各项功能均能正常实现，页面布局正常显示。

2、稳定性

系统后端采用 Spring Boot 框架，数据库使用 MySQL，整体架构成熟稳定。系统支持多用户同时进行商品浏览、搜索、下单支付等操作，WebSocket 即时通讯模块支持多用户实时聊天。系统在并发访问情况下能够保持长时间稳定运行，不会出现数据丢失或服务崩溃的情况。

3、响应时间

系统采用前后端分离架构，前端页面通过 Vue Router 实现路由跳转，响应迅速。商品列表加载、搜索查询、订单创建等核心操作的响应时间控制在 1-2 秒内，聊天消息通过 WebSocket 实时推送，延迟极低，保证用户流畅的交互体验。

4、安全性

系统采用 JWT 进行用户身份认证和授权，接口访问需携带有效 Token。密码采用 BCrypt 加密存储，防止敏感信息泄露。支付宝支付集成采用官方 SDK，支付流程安全可靠。文件上传功能对文件类型和大小进行限制，防止恶意文件上传。

5、可扩展性

系统采用模块化设计，前后端分离架构便于独立扩展。后端 RESTful API 设计规范，新增功能模块只需添加对应接口即可。数据库设计预留扩展字段，便于后续功能升级和数据量增长。

2.2.5 数据需求分析

用户模块：用户注册时输入邮箱、密码（BCrypt 加密存储）、昵称等信息；用户登录时生成 JWT Token；用户修改个人信息时更新昵称、头像等数据。

商品模块：用户发布商品时输入商品名称、价格、描述、商品分类、商品状态（在售/已售/下架）等信息；商品相册存储图片路径；系统记录商品浏览量、收藏量等统计数据。

订单模块：用户创建订单时生成订单号、买家 ID、卖家 ID、商品 ID、订单金额、收货地址、订单状态（待付款/待发货/已完成/已取消）等信息。

聊天模块：用户发送消息时生成会话 ID、买卖双方 ID、消息内容、消息类型（文本/图片）、发送时间、是否已读等信息；WebSocket 连接状态数据。

地址模块：用户添加收货地址时输入收货人姓名、联系电话、省市区、详细地址、是否默认地址等信息。

支付模块：用户发起支付时生成支付订单号、支付金额、支付宝交易号、支付状态、支付时间、回调通知数据等信息。

2.2.6 接口需求分析

1、用户接口

用户携带 JWT Token 通过 RESTful API 接口访问数据。可以完成用户注册、登录、发送验证码、获取当前用户信息、根据用户 ID 获取用户信息、修改个人信息、上传头像、重置密码等操作，返回用户邮箱、昵称、头像、积分等数据。

2、商品接口

用户通过 RESTful API 接口访问数据。可以完成商品列表查询、根据 ID 获取商品详情、发布商品、上传商品图片、修改商品状态、删除商品、搜索商品等操作，返回商品名称、价格、描述、分类、商品状态、相册图片路径、浏览量、收藏量等数据。

3、订单接口

用户携带 JWT Token 通过 RESTful API 接口访问数据。可以完成创建订单、查询订单列表、根据 ID 获取订单详情、更新订单状态等操作，返回订单号、买家 ID、卖家 ID、商品 ID、订单金额、收货地址、订单状态等数据。

4、聊天接口

用户携带 JWT Token 通过 RESTful API 接口访问数据，实时消息通过 WebSocket 协议传输。可以完成会话列表查询、消息列表查询、创建会话、发送消息（支持文本和图片）、标记已读、删除会话等操作，返回会话 ID、买卖双方 ID、消息内容、消息类型、发送时间、是否已读等数据。

5、地址接口

用户携带 JWT Token 通过 RESTful API 接口访问数据。可以完成收货地址列表查询、添加收货地址、删除收货地址等操作，返回收货人姓名、联系电话、省市区、详细地址、是否默认地址等数据。

6、支付接口

用户携带 JWT Token 通过 RESTful API 接口访问数据。可以完成支付宝预下单创建支付订单、支付异步回调通知、同步跳转处理、支付状态查询等操作，返回支付订单号、支付金额、支付宝交易号、支付状态、支付时间等数据。

2.2.7 将来可能提出的需求分析

1、页面美化和用户体验优化：优化商品展示布局，提升页面交互体验，增加暗黑模式支持。

2、智能推荐功能：基于用户浏览历史、收藏记录和购买行为，实现个性化商品推荐，提高用户购买转化率。

3、移动端适配：开发响应式布局或微信小程序版本，支持手机端用户随时随地浏览商品、发布闲置和进行即时通讯。

4、信用评价系统完善：增加用户信用评分机制，基于交易记录、评价内容、履约情况等综合评估用户信用等级。

5、数据分析功能：增加商品浏览量、收藏量、成交率等数据统计分析，为卖家提供经营决策支持。

3 系统设计

3.1 架构设计

本二手交易平台采用前后端分离架构，主要由客户端、业务层、后台服务应用层、数据层、数据库、运行环境之间的关系组成。系统架构图如图 3.1 所示。

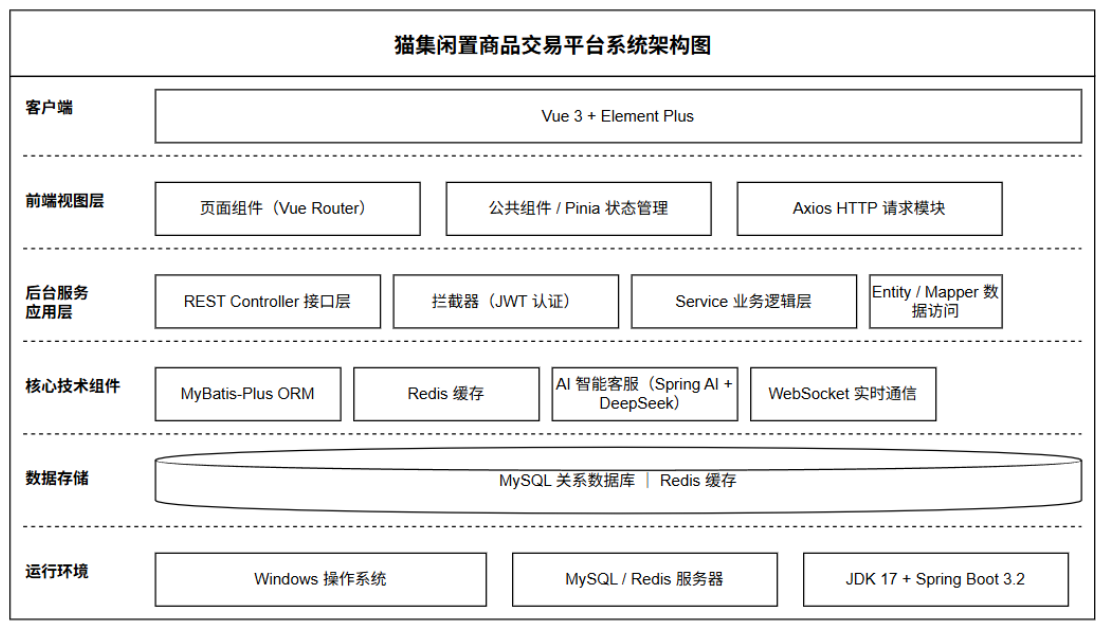


图 3.1 系统架构图

3.2 功能模块设计

本系统主要包含七大功能模块：用户模块、商品模块、订单模块、聊天模块、收货地址模块、支付模块和 AI 智能客服模块。

用户模块负责用户的注册、登录、第三方登录、密码重置和个人信息管理等功能，是用户使用系统的基础。

商品模块是系统的核心模块，提供商品发布、浏览、搜索、分类展示、详情查看和收藏等功能，支撑二手交易的核心业务流程。

订单模块负责订单的创建、支付、状态管理和评价等功能，连接买家和卖家完成交易闭环。

聊天模块提供买卖双方之间的即时通讯功能，支持文本消息、图片发送和未读消息标记，促进交易沟通。

地址模块管理用户的收货地址信息，为订单配送提供地址支持。

支付模块集成支付宝支付功能，提供支付下单、状态查询和异步回调处理等功能，保障交易资金安全。

AI 智能客服模块基于 Spring AI 框架集成 DeepSeek 大模型，提供流式对话与智能工具调用能力。用户可与 AI 助手实时交互。模块具有 Function Calling 机制，AI 可自动调用订单查询等业务工具，结合会话记忆实现上下文连续对话，为用户提供智能化的购物咨询和售后服务。

4 系统实现

4.1 开发环境

本系统采用了前后端分离架构体系，基于 Spring Boot 和 Vue.js 开发，数据库选用 MySQL 开源数据库，系统开发环境如表 4.1 所示。

表 4.1 系统开发环境

硬件环境	软件环境
CPU：Intel Core i5 及以上	操作系统：Windows 10/11
内存：8GB 及以上	数据库：MySQL 8.0
硬盘：256GB 及以上	JDK 版本：JDK 17
浏览器：Chrome/Edge/Firefox	Web 服务器：Spring Boot 3.2.6 内嵌 Tomcat
开发环境：IntelliJ IDEA / Trae	前端框架：Vue.js 3.5.31 + Vite 8.0.3
	UI 组件库：Element Plus 2.13.6
	状态管理：Pinia 3.0.4
	路由管理：Vue Router 5.0.4
	构建工具：Maven + Node.js 20.19.0

4.2 功能模块实现

4.2.1 商品模块

商品模块是系统的核心功能之一，用户在前端浏览\发布\收藏\分类\搜索商品并查看商品详情等操作时，这些商品操作被作为请求发送给后端 CommodityController。后端收到前端传来的商品信息后，进行数据校验和逻辑处理，然后调用 CommodityService 进行业务逻辑处理。Service 层通过 CommodityMapper 接口与 MySQL 数据库进行交互，CommodityMapper 继承 MyBatis-Plus 的 BaseMapper，提供通用的 CRUD 操作。Mapper 层执行 SQL 语句完成数据库操作后，将结果返回给 Service 层，Service 层处理完成后将结果返回给 Controller，最终返回给前端。前端根据返回的信息更新页面显示。

商品发布时，系统自动生成商品 ID，设置初始状态为"在售"，并记录创建时间。商品描述点击右侧商品美化按钮可调用 AI 来对描述进行美化。商品图片通过 FileUploadUtil 工具类上传到服务器，图片路径保存到 Album 表中。

商品代码模块如图 4.1、4.2、4.3、4.4 所示，实现图如图 4.5、4.6、4.7、4.8

所示:

```
@GetMapping("/getCommodityById")
@Operation(summary = "获取商品详情", description = "通过商品ID获取商品详细信息")
public Result<Commodity> getCommodityById(@RequestParam("commodityId") Integer commodityId) {
    Commodity commodity = commodityService.getById(commodityId);
    if (commodity == null) {
        return Result.error(message: "商品不存在");
    }
    LambdaQueryWrapper<Album> albumWrapper = new LambdaQueryWrapper<>();
    albumWrapper.eq(Album::getCommodityId, commodityId);
    List<Album> albums = albumService.list(albumWrapper);
    commodity.setAlbums(albums);
    return Result.success(message: "获取成功", commodity);
}
```

图 4.1 商品详情代码实现图

```
@PostMapping("/addCommodity")
@Operation(summary = "新增商品", description = "新增闲置商品")
public Result addCommodity(@RequestBody CommodityDTO commodityDTO) {
    if (commodityDTO.getImages() == null || commodityDTO.getImages().length == 0) {
        return Result.error(message: "请上传商品图片");
    }
    Commodity commodity = new Commodity();
    commodity.setUserId(commodityDTO.getUserId());
    commodity.setCommodityName(StringUtil.extractFirstLine(commodityDTO.getCommodityDesc()));
    commodity.setCommodityDesc(commodityDTO.getCommodityDesc());
    commodity.setCommodityType(commodityDTO.getCommodityType());
    commodity.setPrice(commodityDTO.getPrice());
    commodity.setBrand(commodityDTO.getBrand());
    commodity.setUseStatus(commodityDTO.getUseStatus());
    commodity.setStatus(CommodityStatus.ON_SHELF.getCode());
    commodity.setBrowse(browse: 0);
    commodity.setCreateTime(java.time.LocalDateTime.now());
    boolean saved = commodityService.save(commodity);
    if (!saved) {
        return Result.error(message: "商品添加失败");
    }
    for (String imagePath : commodityDTO.getImages()) {
        Album album = new Album();
        album.setCommodityId(commodity.getId());
        album.setPath(imagePath);
        album.setCreateTime(java.time.LocalDateTime.now());
        albumService.save(album);
    }
    return Result.success(message: "商品发布成功");
}
```

图 4.2 发布闲置代码实现图

```
public class FileUploadUtil {

    public static String uploadFile(MultipartFile file, String folder) throws IOException {
        if (file.isEmpty()) {
            throw new IOException(message: "文件不能为空");
        }

        String originalFilename = file.getOriginalFilename();
        String suffix = originalFilename.substring(originalFilename.lastIndexOf("."));
        String newFileName = UUID.randomUUID().toString().replace(target: "-", replacement: "") + suffix;

        String uploadPath = UploadConfig.getUploadPath() + "/" + folder;
        File uploadDir = new File(uploadPath);
        if (!uploadDir.exists()) {
            uploadDir.mkdirs();
        }

        File destFile = new File(uploadPath + "/" + newFileName);
        file.transferTo(destFile);

        return "/upload/" + folder + "/" + newFileName;
    }
}
```

图 4.3 上传商品图片代码实现图

```
@PostMapping("/toggle")
@Operation(summary = "切换收藏状态", description = "收藏/取消收藏商品")
public Result<Map<String, Object>> toggleFavorite(
    @RequestHeader("Authorization") String token,
    @RequestParam("commodityId") Integer commodityId) {
    Integer userId = getUserIdFromToken(token);
    Map<String, Object> result = favoriteTool.toggle(userId, commodityId);
    return Result.success(message: null, result);
}
```

图 4.4 商品收藏状态代码实现图

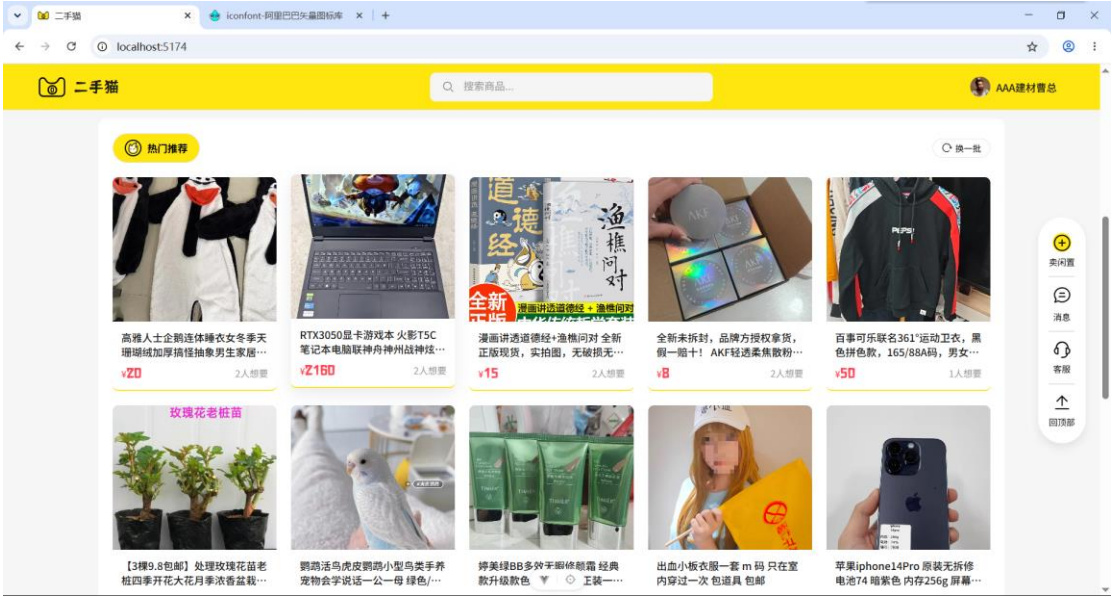


图 4.5 商品模块实现图

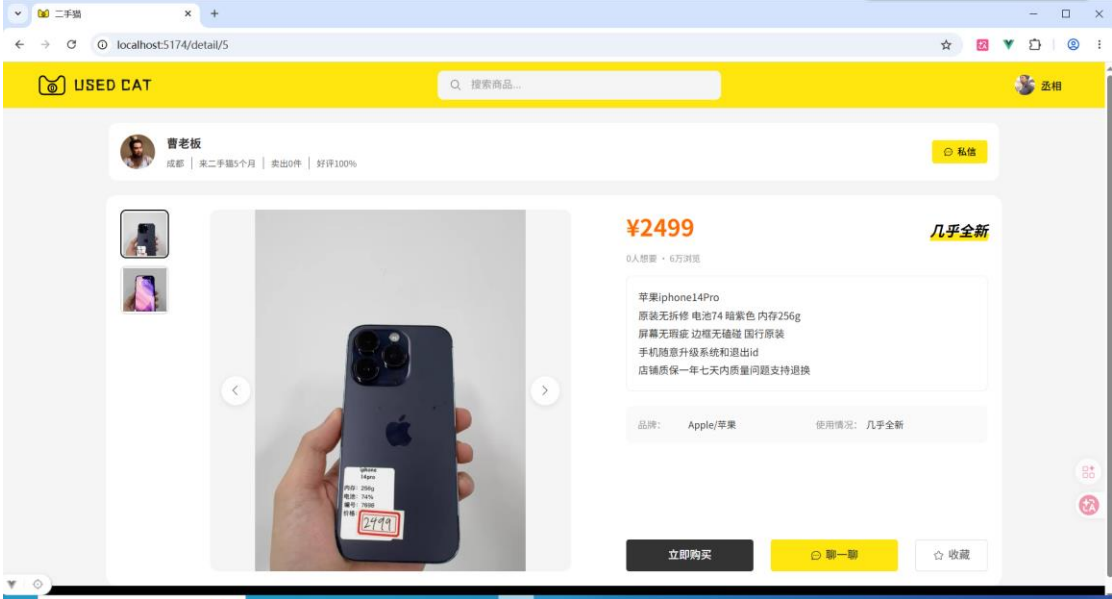


图 4.6 商品详情实现图

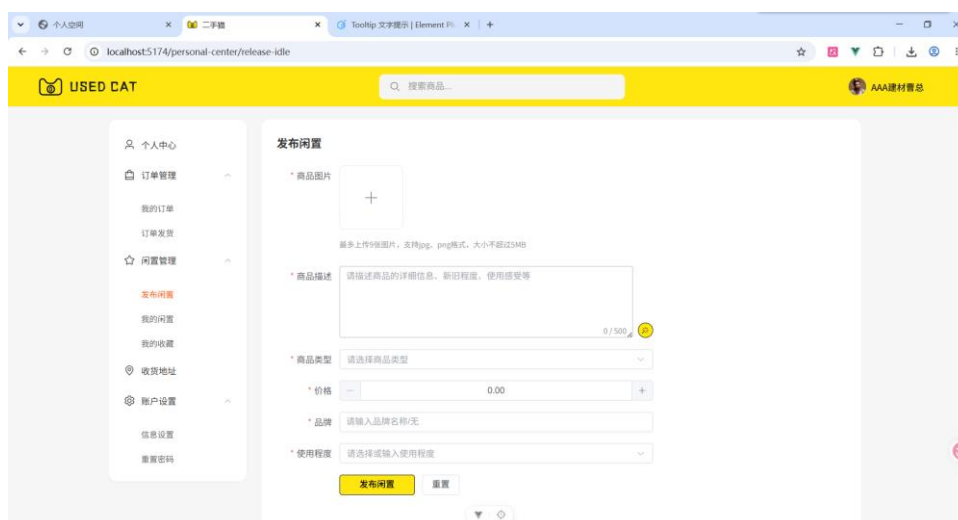


图 4.7 发布闲置实现图

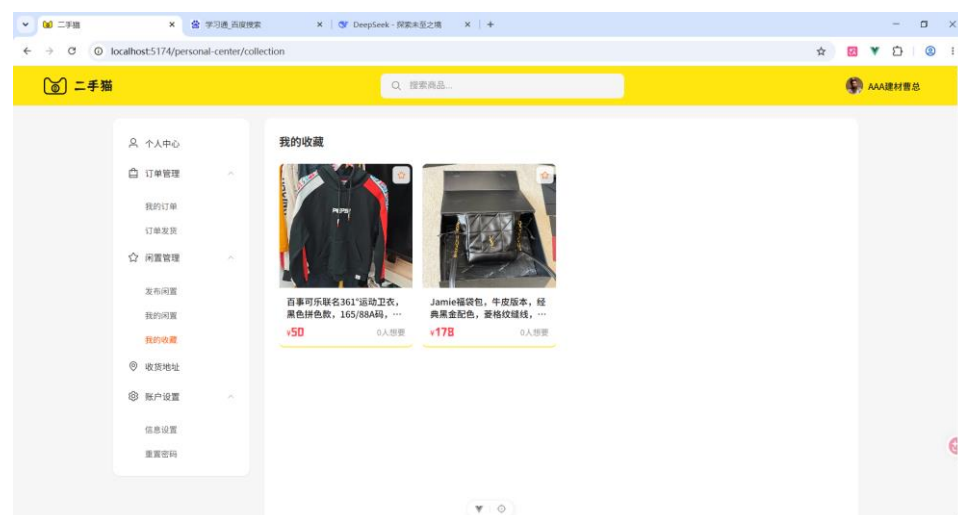


图 4.8 商品收藏页面实现图

4.2.2 用户模块

用户模块负责用户的身份认证和信息管理。用户注册时，前端发送邮箱验证码请求，后端生成 6 位随机验证码存入 Redis（5 分钟有效期），并通过邮件服务发送到用户邮箱。用户提交注册信息后，后端验证邮箱验证码，调用 UserService 进行业务逻辑处理。Service 层通过 UserMapper 接口与 MySQL 数据库进行交互，UserMapper 继承 MyBatis-Plus 的 BaseMapper，提供通用的 CRUD 操作。Mapper 层执行 SQL 语句完成数据库操作后，将结果返回给 Service 层，Service 层使用 BCrypt 加密密码后保存到数据库。登录时，后端验证密码并生成 Token 返回给前端。前端根据返回的信息更新页面显示。

用户认证采用 JWT 无状态认证方案，Token 中包含用户 ID 等基本信息，前端在后续请求中携带 Token 进行身份验证。密码使用 BCrypt 加密存储，确保用户信息安全。

用户模块代码如图 4.9、4.10 所示，实现图如图 4.11 所示：

```

@GetMapping("/sendCode")
@Operation(summary = "发送邮箱验证码", description = "向指定邮箱发送验证码并存入Redis, 5分钟有效")
public Result sendVerificationCode(@RequestParam("email") String email) {    Result is a raw type.
    User existUser = userService.findUserByEmail(email);
    if (existUser != null) {
        return Result.error(message: "该邮箱已被注册");
    }

    String code = String.valueOf((int) ((Math.random() * 9 + 1) * 100000));

    String redisKey = "verify:code:" + email;
    stringRedisTemplate.opsForValue().set(redisKey, code, timeout: 5, TimeUnit.MINUTES);

    mailUtil.sendVerificationCode(email, code);

    return Result.success(message: "验证码已发送请查收邮件, 验证码5分钟内有效, 请勿泄露他人。", data: null);
}

```

图 4.9 发送邮箱验证码代码实现图

```

/**
 * 密码加密
 * @param rawPassword 明文密码
 * @return 加密后的密码
 */
public static String encode(String rawPassword) {
    if (rawPassword == null || rawPassword.isEmpty()) {
        throw new IllegalArgumentException(s: "密码不能为空");
    }
    return ENCODER.encode(rawPassword);
}

```

图 4.10 加密代码实现图

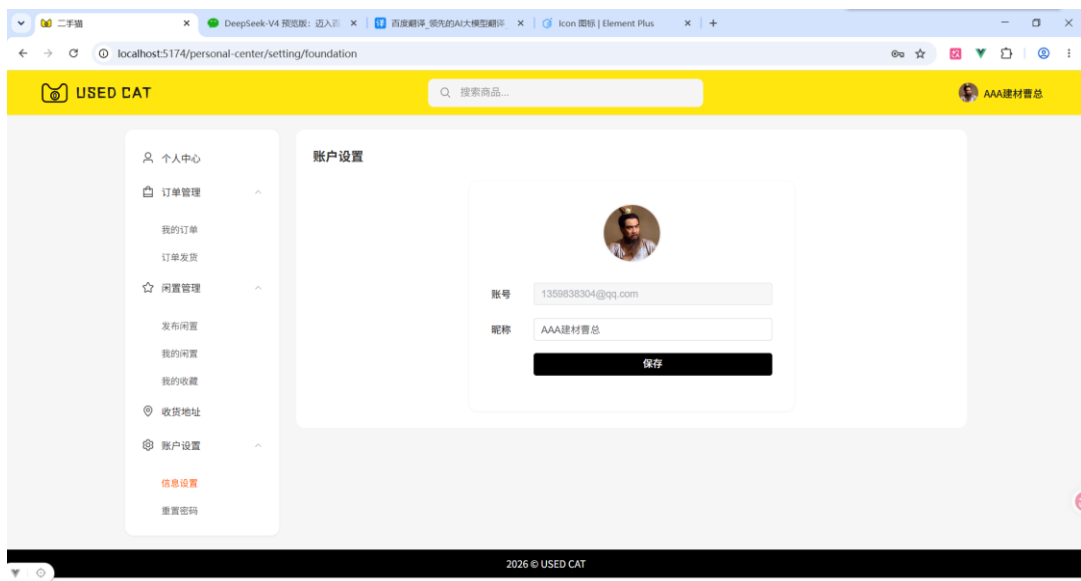


图 4.11 用户设置实现图

4.2.3 订单模块

订单模块负责处理用户的购买流程。用户在前端点击"立即购买"后，前端将

商品信息、收货地址 ID 等数据发送给 OrderController。后端收到请求后，首先根据地址 ID 通过 ReceivingAddressMapper 查询收货地址信息，然后调用 OrderService 进行业务逻辑处理。Service 层通过 OrderMapper 接口与 MySQL 数据库进行交互，OrderMapper 继承 MyBatis-Plus 的 BaseMapper，提供通用的 CRUD 操作。Mapper 层执行 SQL 语句完成数据库操作后，将结果返回给 Service 层，Service 层处理完成后将结果返回给 Controller，最终返回给前端。前端根据返回的信息更新页面显示。

订单创建时，系统自动填充收货人信息、联系电话和详细地址，确保订单信息完整。用户订单下单后可在购买记录点击查看物流查看当前物流信息。用户不想要该商品时也可以点击取消订单。

订单模块代码如图 4.12、4.13 所示，实现图如图 4.14、4.15 所示：

```
@PostMapping("/create")
@Operation(summary = "创建订单", description = "创建新订单")
public Result<Order> createOrder(@RequestBody CreateOrderDTO dto) {
    Order order = new Order();
    order.setOrderNo("ORD" + System.currentTimeMillis());
    order.setUserId(dto.getUserId());
    order.setCommodityId(dto.getCommodityId());
    order.setCommodityName(dto.getCommodityName());
    order.setPrice(dto.getPrice());
    order.setQuantity(dto.getQuantity());
    order.setTotalAmount(dto.getTotalAmount());
    order.setAddressId(dto.getAddressId());
    order.setStatus(0); // 待付款
    order.setCreateTime(LocalDateTime.now());

    ReceivingAddress address = receivingAddressService.getById(dto.getAddressId());
    if (address != null) {
        order.setConsignee(address.getConsignee());
        order.setPhone(address.getPhone());
        order.setAddress(address.getRegion() + address.getAddress());
    }

    boolean success = orderService.save(order);
    if (success) {
        return Result.success("订单创建成功", order);
    }
    return Result.error("订单创建失败");
}
```

图 4.12 创建订单代码实现图

```
@GetMapping("/list")
@Operation(summary = "获取订单列表", description = "根据userId获取订单列表")
public Result<List<Order>> getOrderList(@Parameter(description = "用户id") @RequestParam("userId") Integer userId) {
    List<Order> orders = orderService.lambdaQuery()
        .eq(Order::getUserId, userId)
        .orderByDesc(Order::getCreateTime)
        .list();
    return Result.success(message: "获取订单列表成功", orders);
}
```

图 4.13 获取用户订单代码实现图



图 4.14 订单界面实现图

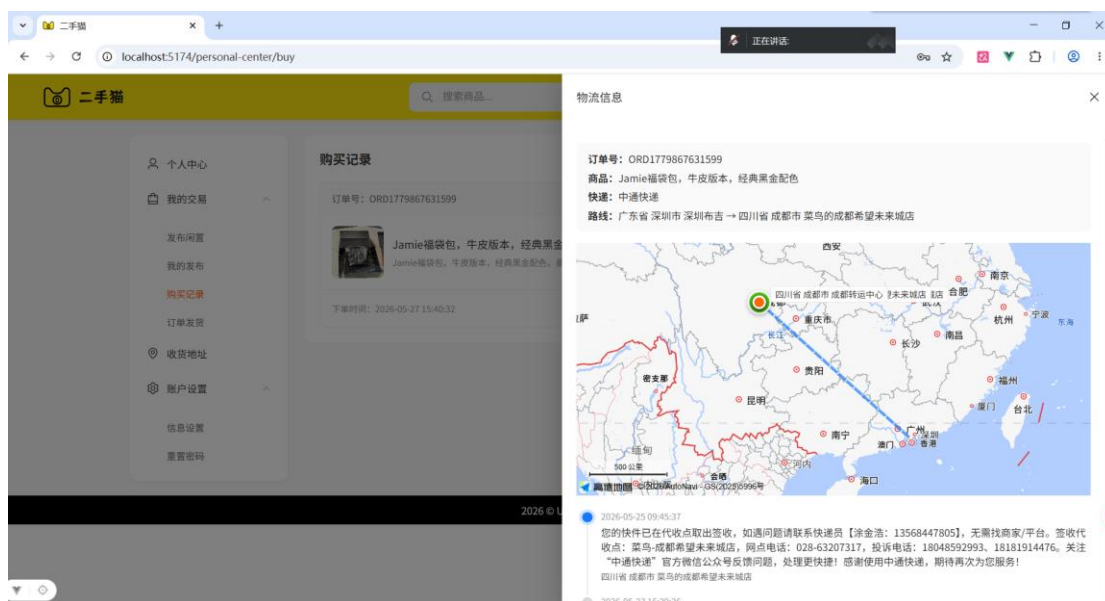


图 4.15 用户订单界面实现图

4.2.4 收货地址模块

收货地址模块管理用户的配送地址信息。用户在前端填写收货人姓名、联系电话、省市区和详细地址后，前端将数据发送给 `ReceivingAddressController`。后端收到请求后，调用 `ReceivingAddressService` 进行业务逻辑处理。`Service` 层通过 `ReceivingAddressMapper` 接口与 `MySQL` 数据库进行交互，`ReceivingAddressMapper` 继承 `MyBatis-Plus` 的 `BaseMapper`，提供通用的 `CRUD` 操作。`Mapper` 层执行 `SQL` 语句完成数据库操作后，将结果返回给 `Service` 层，`Service` 层处理完成后将结果返回给 `Controller`，最终返回给前端。前端根据返回的信息更新页面显示。

收货地址模块代码如图 4.16 所示，实现图如图 4.17 所示：


```
@PostMapping("/addReceivingAddress")
@Operation(summary = "创建收货地址", description = "创建新的收货地址")
public Result<Void> addAddress(@RequestBody AddReceivingAddressDTO dto) {
    ReceivingAddress address = new ReceivingAddress();
    address.setUserId(dto.getUserId());
    address.setConsignee(dto.getConsignee());
    address.setPhone(dto.getPhone());
    address.setRegion(dto.getRegion());
    address.setAddress(dto.getAddress());

    boolean success = receivingAddressService.save(address);
    if (success) {
        return Result.success("创建收货地址成功!", null);
    }
    return Result.error("创建收货地址失败");
}
```

图 4.16 创建收货地址代码实现图

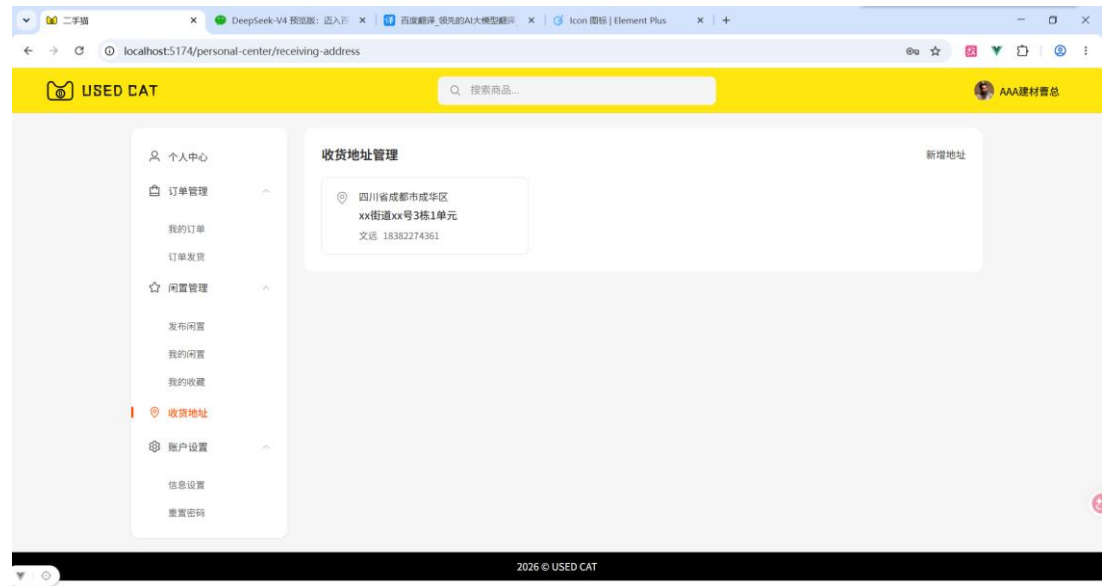


图 4.17 收货地址管理

4.2.5 收货地址模块

支付模块集成支付宝官方 SDK，实现安全的在线支付功能。用户确认订单后，前端调用 AlipayController 的预下单接口，后端通过 AlipayService 调用支付宝 API 生成支付二维码发送给前端展示在界面。用户扫码支付后，支付宝异步通知回调后端 notify 接口，后端验证签名后调用 OrderService 和 CommodityService 进行业务逻辑处理。Service 层通过 OrderMapper 和 CommodityMapper 接口与 MySQL 数据库进行交互，Mapper 层继承 MyBatis-Plus 的 BaseMapper，提供通用的 CRUD 操作。Mapper 层执行 SQL 语句完成数据库操作后，将结果返回给 Service 层，Service 层处理完成后将结果返回给 Controller，最终返回给前端。前端根据返回的信息更新页面显示。

支付回调处理时，系统首先验证支付宝签名确保请求合法性，然后更新订单状态为"已付款"，同时将商品状态更新为"已售出"，防止重复购买。

支付模块代码如图 4.18、4.19 所示，实现图如图 4.20 所示：

```
public class AlipayService {
    * 创建支付宝二维码支付
    * @param outTradeNo 订单号
    * @param totalAmount 订单金额
    * @param subject 订单标题
    * @return 二维码图片
    */
    public String createQrCodePayment(String outTradeNo, String totalAmount, String subject) {
        try {
            AlipayTradePrecreateRequest request = new AlipayTradePrecreateRequest();
            request.setNotifyUrl(alipayConfig.getNotifyUrl());
            Map<String, String> bizMap = new HashMap<>();
            bizMap.put(key: "out_trade_no", outTradeNo);
            bizMap.put(key: "total_amount", totalAmount);
            bizMap.put(key: "subject", subject);
            request.setBizContent(objectMapper.writeValueAsString(bizMap));
            AlipayClient alipayClient = getAlipayClient();
            AlipayTradePrecreateResponse response = alipayClient.execute(request);
            if (response.isSuccess()) {
                log.info(format: "二维码创建成功, outTradeNo: {}, qrCode: {}", outTradeNo, response.getQrCode());
                return response.getQrCode();
            } else {
                log.error(format: "二维码创建失败: {}, subCode: {}, subMsg: {}",
                    response.getMsg(), response.getSubCode(), response.getSubMsg());
                return null;
            }
        } catch (AlipayApiException e) {
            log.error(format: "二维码支付异常: {}", e.getMessage(), e);
            return null;
        } catch (JsonProcessingException e) {
            log.error(format: "JSON序列化异常: {}", e.getMessage(), e);
            return null;
        }
    }
}
```

图 4.18 创建支付宝支付二维码代码实现图

```
@PostMapping("/notify")
@Operation(summary = "支付回调", description = "支付宝异步通知回调")
public String notify(HttpServletRequest request) {
    Map<String, String> params = new HashMap<>();
    Map<String, String[]> requestParams = request.getParameterMap();
    for (String name : requestParams.keySet()) {
        String[] values = requestParams.get(name);
        String valueStr = "";
        for (int i = 0; i < values.length; i++) {
            valueStr = (i == values.length - 1) ? valueStr + values[i] : valueStr + values[i] + ",";
        }
        params.put(name, valueStr);
    }

    log.info(format: "收到支付宝回调, params: {}", params);
    if (alipayService.verifySignature(params)) {
        String tradeStatus = params.get(key: "trade_status");
        String outTradeNo = params.get(key: "out_trade_no");
        String tradeNo = params.get(key: "trade_no");
        String totalAmount = params.get(key: "total_amount");
        if ("TRADE_SUCCESS".equals(tradeStatus) || "TRADE_FINISHED".equals(tradeStatus)) {
            log.info(format: "支付成功, outTradeNo: {}, tradeNo: {}, totalAmount: {}", outTradeNo, tradeNo, totalAmount);
            Order order = orderService.lambdaQuery()
                .eq(Order::getOrderNo, outTradeNo)
                .one();
            if (order != null) {
                if (order.getStatus() != null && order.getStatus() != 1) {
                    order.setStatus(status: 1);
                    order.setPayMethod(payMethod: "alipay");
                    order.setTradeNo(tradeNo);
                    order.setPayTime(LocalDateTime.now());
                    order.setUpdateTime(LocalDateTime.now());
                }
            }
        }
    }
}
```

图 4.19 支付回调代码实现图

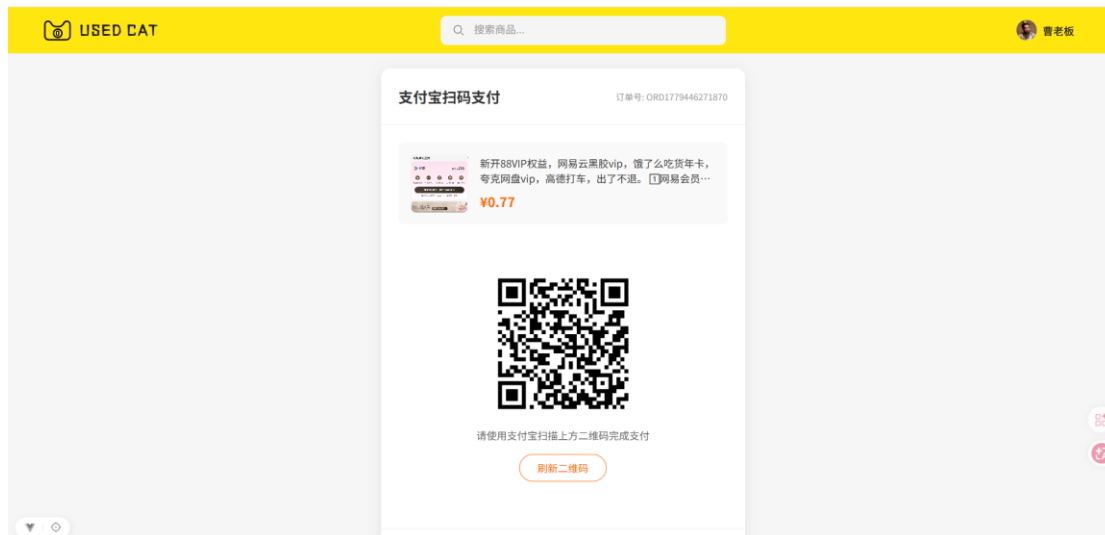


图 4.20 支付宝扫码支付界面实现图

4.2.6 聊天模块

聊天模块是系统的创新功能，采用 WebSocket 实现实时双向通信。用户在商品详情页点击"聊一聊"或私信按钮，前端建立 WebSocket 连接并携带 JWT Token 进行身份验证。后端 ChatWebSocketHandler 处理连接建立、消息收发和连接关闭等事件。用户发送消息时，消息通过 WebSocket 实时推送到对方客户端，同时调用 MessageService 进行业务逻辑处理。Service 层通过 MessageMapper 和 ConversationMapper 接口与 MySQL 数据库进行交互，Mapper 层继承 MyBatis-Plus 的 BaseMapper，提供通用的 CRUD 操作。Mapper 层执行 SQL 语句完成数据库操作后，将结果返回给 Service 层，Service 层处理完成后将结果返回给 Controller，最终返回给前端。前端根据返回的信息更新页面显示。

WebSocket 连接建立时，通过 JwtUtil 解析 Token 获取用户 ID，确保通信安全性。消息支持文本和图片两种类型，未读消息通过 unreadCount 字段标记。

支付模块代码如图 4.21、4.22、4.23 所示，实现图如图 4.24 所示：

```
/**
 * 向指定用户发送消息
 * @param userId 用户 ID
 * @param message 要发送的消息
 */
public void sendToUser(Integer userId, Message message) {
    WebSocketSession session = userSessions.get(userId);
    if (session == null || !session.isOpen()) {
        log.warn(format: "WebSocket 推送失败: 用户 {} 不在线或连接已关闭, 消息: {}", userId, message.getContent());
        return;
    }
    try {
        String json = objectMapper.writeValueAsString(message);
        session.sendMessage(new TextMessage(json));
        log.info(format: "WebSocket 推送成功: userId={}, content={}", userId, message.getContent());
    } catch (Exception e) {
        log.error(format: "WebSocket 推送异常: userId={}, error={}", userId, e.getMessage());
    }
}
```

图 4.21 指定发送用户代码实现图

```

@PostMapping("/message/send")
@Operation(summary = "发送消息", description = "发送聊天消息")
public Result sendMessage(@RequestHeader("Authorization") String token,
    @RequestBody Map<String, Object> body) {
    Integer senderId = getUserIdFromToken(token);
    Integer conversationId = (Integer) body.get(key: "conversationId");
    Integer receiverId = (Integer) body.get(key: "receiverId");
    String content = (String) body.get(key: "content");
    String messageType = body.getOrDefault(key: "messageType", defaultValue: "text").toString();

    String lastMessage = "image".equals(messageType) ? "[图片]" : content;

    Message message = new Message();
    message.setConversationId(conversationId);
    message.setSenderId(senderId);
    message.setReceiverId(receiverId);
    message.setContent(content);
    message.setMessageType(messageType);
    message.setIsRead(isRead: 0);
    message.setCreateTime(LocalDateTime.now());
    messageService.save(message);

    conversationService.update(
        new LambdaUpdateWrapper<Conversation>()
            .eq(Conversation::getConversationId, conversationId)
            .set(Conversation::getLastMessage, lastMessage)
            .set(Conversation::getLastTime, LocalDateTime.now())
    );
};

```

图 4.22 发送消息接口(上) 代码实现图

```

Conversation receiverConv = conversationService.getOne(
    new LambdaQueryWrapper<Conversation>()
        .eq(Conversation::getUserId, receiverId)
        .eq(Conversation::getTargetUserId, senderId)
);
if (receiverConv != null) {
    if (receiverConv.getStatus() != null && receiverConv.getStatus() == 0) {
        conversationService.update(
            new LambdaUpdateWrapper<Conversation>()
                .eq(Conversation::getConversationId, receiverConv.getConversationId())
                .set(Conversation::getStatus, 1)
        );
    }
    conversationService.update(
        new LambdaUpdateWrapper<Conversation>()
            .eq(Conversation::getConversationId, receiverConv.getConversationId())
            .set(Conversation::getLastMessage, lastMessage)
            .set(Conversation::getLastTime, LocalDateTime.now())
            .setSql("unread_count = unread_count + 1")
    );
}
websocketHandler.sendToUser(senderId, message);
if (receiverConv != null) {
    Message receiverMsg = new Message();
    receiverMsg.setMessageId(message.getMessageId());
    receiverMsg.setConversationId(receiverConv.getConversationId());
    receiverMsg.setSenderId(senderId);
    receiverMsg.setReceiverId(receiverId);
    receiverMsg.setContent(content);
    receiverMsg.setMessageType(messageType);
}

```

图 4.23 发送消息接口(下)代码实现图

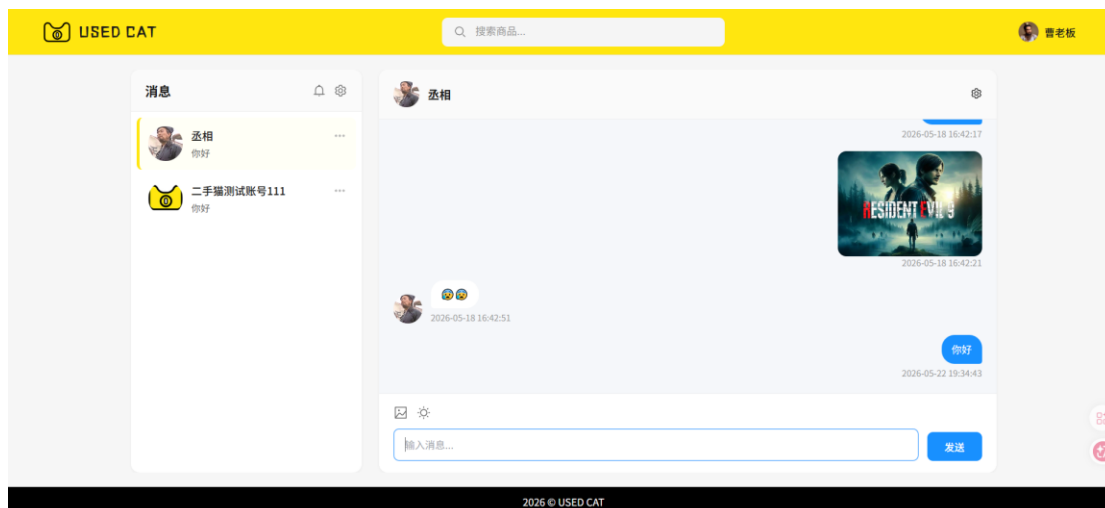


图 4.24 聊天界面实现图

4.2.7 AI 智能客服模块

AI 智能客服模块基于 Spring AI 框架集成 DeepSeek 大模型，提供购物咨询与售后问答服务。用户通过前端 LiaoKit 聊天组件与“二手猫”AI 助手对话，支持 SSE 流式响应、Markdown 渲染和打字机效果。

模块使用 Spring AI 的 ChatClient 与 DeepSeek 模型通信，在 AiConfig 中创建 ChatClient Bean。对话由 UsedCatAgent 统一编排：先通过 ChatMemoryTool 获取或创建会话，加载历史记录作为上下文，再调用 ChatClient 发起流式请求，生成内容以 Flux<String>返回前端，回复自动持久化至数据库。

模块内置 Function Calling 能力，通过 @Tool 注解将订单查询注册为 AI 可调用工具。当用户询问“我的订单”等问题时，DeepSeek 识别意图并返回 tool_call 请求，Spring AI 执行对应方法后回传结果进行二次推理，生成自然语言回复。ToolContext 在工具调用链中安全传递用户 ID 等上下文信息。

支付模块代码如图 4.25、4.26、4.27 所示，实现效果见图 4.28。

```
@GetMapping("/customerServiceInit")
@Operation(summary = "初始化客服会话", description = "查找该用户客服会话，不存在则创建")
public Result<Session> customerServiceInit(@RequestParam("userId") Integer userId) {
    Session session = chatMemoryTool.getOrCreateSession(userId);
    return Result.success(message: "获取会话成功", session);
}

@GetMapping(value = "/customerServiceChat", produces = "text/stream;charset=utf-8")
@Operation(summary = "客服流式对话", description = "智能客服流式对话，自动按userId查找或创建会话")
public Flux<String> customerServiceChat(
    @RequestParam("message") String message,
    @RequestParam("userId") Integer userId
) {
    return usedCatAgent.streamChat(message, userId);
}
```

图 4.25 客服接口实现图

```

@Tool(description = "查询当前登录用户的订单列表。可以根据订单状态进行过滤。当用户询问“我的订单”、“订单状态”时调用此工具。")
public String getUserOrders(
    @ToolParam(description = "订单状态 0=待付款 1=已付款等待卖家发货 2=已发货运输中 3=已完成。传null或不传则查询所有订单。", required = false)
    Integer status,
    ToolContext toolContext) {

    Integer userId = (Integer) toolContext.getContext().get(key: "userId");
    if (userId == null) {
        return "无法获取当前用户信息，请重新登录后再试。";
    }

    var query = orderService.lambdaQuery()
        .eq(Order::getUserId, userId)
        .orderByDesc(Order::getCreateTime);

    if (status != null && status >= 0 && status <= 3) {
        query.eq(Order::getStatus, status);
    }

    List<Order> orders = query.list();

    if (orders.isEmpty()) {
        String statusLabel = getStatusLabel(status);
        return "您当前没有" + statusLabel + "订单。";
    }
}

```

图 4.26 订单查询工具代码实现图

```

public Flux<String> streamChat(String message, Integer userId) {
    Long sessionId = chatMemoryTool.getOrCreateSession(userId)
        .getSessionId().longValue();

    History userHistory = chatMemoryTool.saveUserMessage(message, sessionId);

    List<Message> chatMessages = chatMemoryTool.loadRecentHistory(sessionId, userHistory.getId())
        .stream()
        .map(h -> "user".equals(h.getRole())
            ? new UserMessage(h.getContent())
            : new AssistantMessage(h.getContent()))
        .collect(Collectors.toList());

    StringBuilder[] builders = {new StringBuilder()};

    return chatClient
        .prompt(aiPromptConfig.resolve(aiPromptConfig.getCustomerService()))
        .user(message)
        .messages(chatMessages)
        .tools(orderQueryTools)
        .toolContext(Map.of(k1: "userId", userId))
        .stream()
        .content()
        .doOnNext(s -> builders[0].append(s))
        .doOnComplete(() ->
            chatMemoryTool.saveAssistantMessage(builders[0].toString(), sessionId)
        )
        .doOnError(e -> {
            log.error(format: "AI 对话异常: {}", e.getMessage());
            String partial = builders[0].toString();
            if (!partial.isEmpty()) {
                chatMemoryTool.saveAssistantMessage(partial, sessionId);
            }
        })
        .onErrorResume(e ->
            Flux.just(data: "\n\n[抱歉，网络出现波动，请稍后重试]")
        );
}

```

图 4.27 agent 核心代码实现图



图 2.28 AI 智能客服界面实现图